

Introduction to Robotics, Lecture 6

Programming real manipulators

Path planning

Trajectory planning

MIPT

Center for Cognitive Modeling

Why are two target poses not enough?

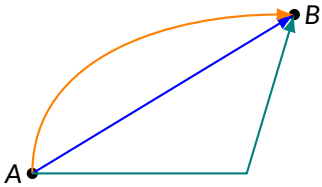
The same start and goal

Which path should the robot follow?

How fast should it move?

What happens between the targets?

A target does not specify the complete motion.



From a task to motion

Task → path → timing law → execution

Teach, store, repeat

Teaching a motion

Guide the robot or move it with a teach pendant. Store positions or a recorded motion.

Playback

Repeat the stored sequence; interpolate between taught positions.

$$q_0 \longrightarrow q_1 \longrightarrow \cdots \longrightarrow q_n$$

- The program assumes that the scene stays as taught.
- A moved workpiece does not automatically change the program.
- Task feedback requires additional sensing and logic.

Internal motion feedback and adaptation to the scene are different levels.

From taught points to a robot program

A sequence of actions	Industrial robot languages
Move to a pose. Perform an operation. Read an input. Choose the next action.	KUKA: KRL Universal Robots: URScript

Conditions and loops make a program more than a list of points.

A program combines motion commands with interaction with other equipment.

Motion primitives still leave interpolation and execution to the controller.

The pendant and an external computer

Program and execute

PolyScope: build and run a program on the pendant.
URScript: express motion and program logic for the robot controller.

Exchange and integrate

RTDE: exchange robot state and configured input/output data.
ROS driver: connect the robot to external robotics software.

Why connect a computer?

Log measurements, calculate a plan, use sensors, or implement task feedback.

External computation ⟷ communication interface ⟷ controller program

A communication interface is not a control law.

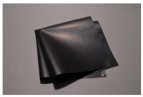
UR: overview of client interfaces

"2" Верификация преобразователя силы

Измерить характеристики материала для случаев, когда **площадь приложения силы меньше**, чем **площадь активной части** сенсора.

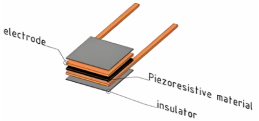
Входные данные: показания разработанного датчика и значение реально приложенной нагрузки.

Выходные данные: разница между нормализованным значением с датчика и реальной нагрузкой.



Материал Velostat





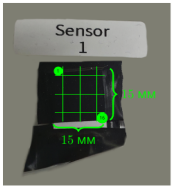
Преобразователь силы

Верификация преобразователя силы

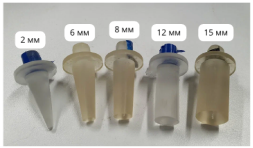
Алгоритм: 2 эксперимента:

1. Статический — прикладывается статический груз с размером в сенсор для калибровки.
2. Динамический — чувствительная область представляется в виде сетки 4 × 4. Происходит **касание** каждой области **с одинаковым давлением**, но разной площадью контакта.

Предположения: 1) материал обладает вязко-эластичными свойствами, поэтому надо учитывать гистерезис. **Допустимая ошибка**: 10%



Поверхность как 4 × 4 сетка



Все насадки

Статический эксперимент

$$V_{out} = V_0 + p[\,k_p + k_e(1 - e^{\frac{-(t-t_0)}{\tau_{res}}})](1 - e^{-\frac{A}{p}}) \quad (15)$$

$$k_p = A_1 e^{-A_2 p} \quad (16)$$

$$\tau_{res} = B_0 + B_1 e^{-\frac{p}{B_2}} \quad (17)$$

Где V_0 – начальное напряжение,

p – приложенное давление,

A_i, B_i, τ_{res}, k_i искомые параметры,

t – текущее время, t_0 – время начала нажатия.

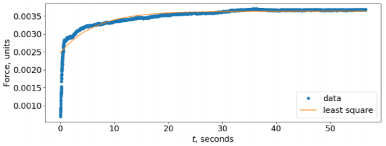
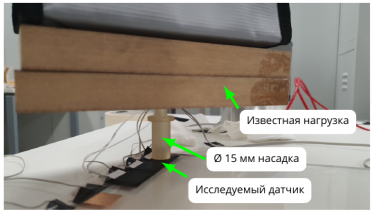
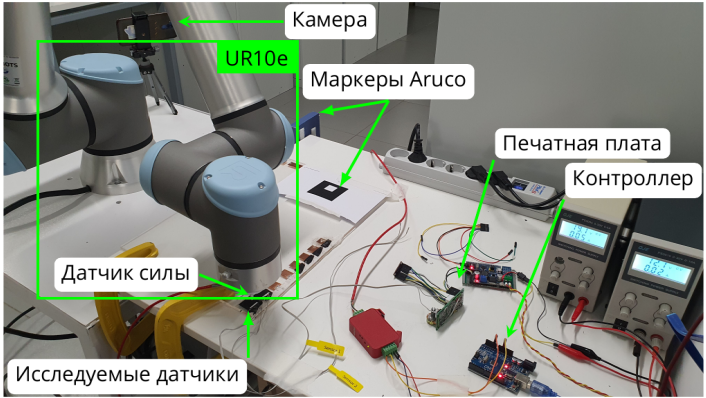


График регрессии

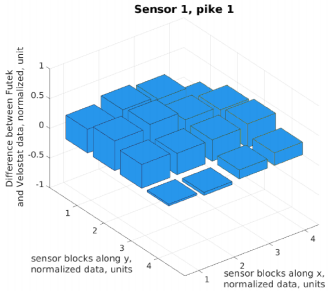


Экспериментальная установка

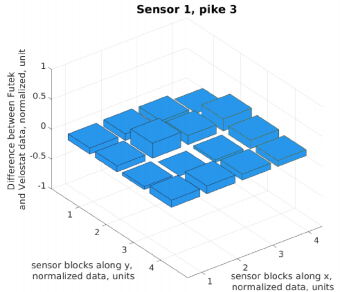
Динамический эксперимент: Установка



Результаты динамического эксперимента



2 мм диаметр насадки



8 мм диаметр насадки

Одинаковые данные, когда площадь нажатия превышает 25% от площади датчика

From task to trajectory

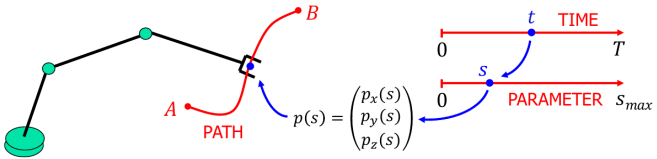
TRAJECTORY = GEOMETRIC PATH + TIMING LAW

TRAJECTORY: of motion $p_d(t)$ (or $q_d(t)$); of interaction $F_d(t)$ – not pursued here

GEOMETRIC PATH: parameterized by s: $p = p(s)$ (any s or the arc length)

TIMING LAW: describes the time evolution of $s = s(t)$

$p(s(t))$

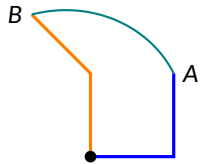


example: TASK planner provides A, B
TRAJECTORY planner generates $p(t)$

Two spaces for planning the same transfer

Joint space

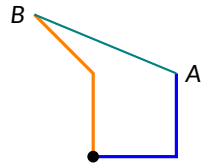
$$q(s) = q_0 + s(q_f - q_0)$$



A joint-space line usually produces a curved TCP path. Compute it with FK.

Cartesian space

$$p(s) = p_0 + s(p_f - p_0)$$



A TCP line needs IK along the path and a continuous solution branch.

KRL language

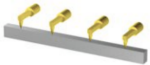
- typical motion primitives



PTP motion
(point-to-point, linear
in joint space)

LIN motion
(linear in
Cartesian space)

CIRC motion
(circular in
Cartesian space)



CONST orientation

end-effector
orientation



PTP motion
(linear in RPY angles)

Robotics 1 *7*

KUKA example. UR equivalents are introduced on the next slide.

Motion primitives on Universal Robots

From the KUKA example to UR

PTP ⟶ **MoveJ**: joint-space motion.
LIN ⟶ **MoveL**: linear TCP motion.

These match the purpose of the motion; interpolation details depend on the controller.

Programming the UR

URScript: movej, movel.
MoveP / CircleMove: process and circular motion.

The previous RPY example belongs to KUKA; it is not an orientation rule for UR.

UR: motion commands

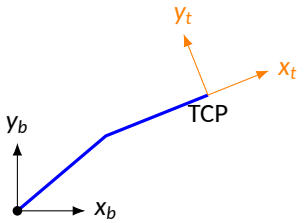
Coordinates, TCP and manual movement

What do we move?

Joint jogging: change an individual joint angle.
TCP jogging: move the tool pose in a selected coordinate frame.

Freedrive

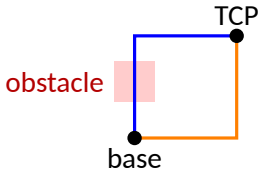
Guide the arm by hand to teach a pose. Saving waypoints stores poses, not the complete hand-guided path.



The same coordinate increment means different physical directions in **base** and **tool** frames.

UR: Freedrive

The TCP is clear. What about the elbow?



Same TCP position, different arm configurations.

Check the whole robot

All links, the tool and the environment contribute to collision constraints.

- A clear TCP path is insufficient.
- Different IK solutions have different clearance.
- Self-collision may also matter.

Obstacles in configuration space

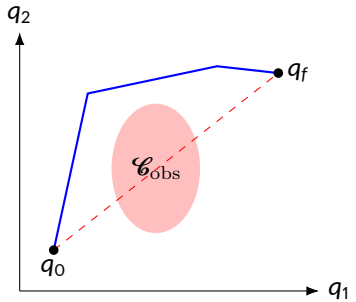
One point (q_1, q_2) specifies the entire RR arm.

Free configurations

$$\mathscr{C}_{\text{free}} = \mathscr{C} \setminus \mathscr{C}_{\text{obs}}$$

Find a path from q_0 to q_f through the free region.

An obstacle in the physical workspace induces forbidden *configurations*.

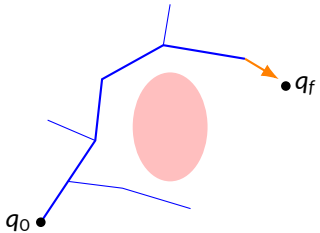


Schematic forbidden region, not a calculated collision map.

Finding a path: the idea of RRT

1. Sample a configuration.
2. Find the nearest tree node.
3. Extend the tree towards the sample.
4. Validate the new motion.
5. Repeat until a connection to the goal is found.

Collision checking is needed along edges, not only at nodes.



A geometric path still needs simplification, timing and execution.

OMPL: sampling-based planners

A modern pipeline: cuRobo

Inputs

Robot model, collision geometry, scene, current state and target pose.

Planning components

IK, collision checking, geometric planning and trajectory optimization.

GPU computation evaluates many candidates in parallel.

From a model to motion

Model + scene + goal

↓

Motion generation

↓

Validate and execute

NVIDIA cuRobo: documentation and examples

The path is chosen: add a timing law

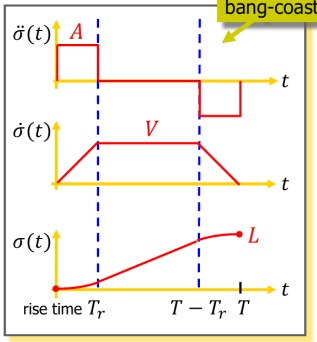
Rest-to-rest motion	The chain rule
$s(0) = 0, \quad s(T) = 1$ $\dot{s}(0) = \dot{s}(T) = 0$	$\dot{q} = q'(s)\dot{s}$ $\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}$

Speed and acceleration limits constrain how quickly we can traverse the path.

A common $s(t)$ keeps all coordinates on the chosen path.

Constant speed cannot start instantaneously with bounded acceleration.

Timing law with trapezoidal speed - 1



given*: L, V, A
find: T_r, T

$V\ (T - T_r) = L$ ← = area of the speed profile

$$T_r = \frac{V}{A}$$
$$T = \frac{A\,L + V^2}{AV}$$

a "coast" phase exists iff $L > V^2/A$

otherwise …

triangular speed (bang-bang) profile with

$\max\ \dot{\sigma} = \bar{V} = A\,T_r = A \cdot T/2 \le V$

$\bar{V} \cdot T/2 = L$ (= area)

$$T = \sqrt{4L/A}$$

* = other input data combinations are possible (see textbook Sect. 4.4.2, p. 217)

When the trapezoid becomes a triangle

If $D < V^2/A$, the robot cannot reach V before braking.

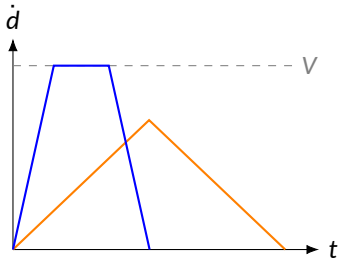
$$V_p = \sqrt{AD} < V$$

$$t_r = \sqrt{\frac{D}{A}}, \qquad T = 2\sqrt{\frac{D}{A}}$$

In both cases:

$$V_p = \min\left(V, \sqrt{AD}\right)$$

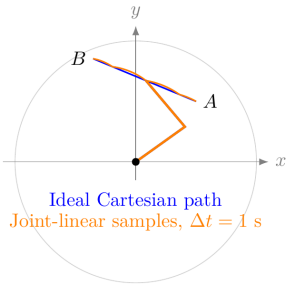
At $D = V^2/A$, the plateau has zero duration.

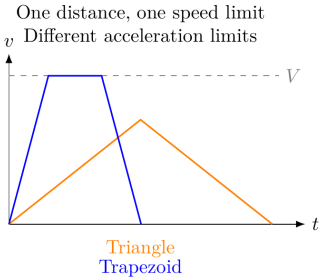


Same D and V: increase A to change a triangle into a trapezoid.

$$A_{\text{crit}} = \frac{V^2}{D}$$

Live example: change the acceleration





RR: $A = (1, 1)$, $B = (-\sqrt{2}/2, 1 + \sqrt{2}/2)$ m. Joint space: $V = 60°/\mathrm{s}$; compare $A = 20$ and $120°/\mathrm{s}^2$.

Problem: move an RR arm between two configurations

Two links: $l_1 = l_2 = 1\,\text{m}$.

$$q_0 = (0°, 90°), \quad q_f = (90°, 45°)$$

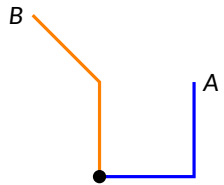
For **each** joint:

$$|\dot{q}_i| \leq 60°/\text{s}, \qquad |\ddot{q}_i| \leq 60°/\text{s}^2$$

Start and finish at rest. Follow a straight path in joint space.

Find

A common timing law and the total duration.



Given FK:

$$p = \begin{bmatrix} \cos q_1 + \cos(q_1 + q_2) \\ \sin q_1 + \sin(q_1 + q_2) \end{bmatrix}$$

Synchronize the joints with one progress variable

$$\Delta q = (90^\circ, -45^\circ)$$

$$q_1 = 90^\circ s, \qquad q_2 = 90^\circ - 45^\circ s$$

$$\dot{q}_i = \Delta q_i \dot{s}, \qquad \ddot{q}_i = \Delta q_i \ddot{s}$$

Preserve the joint-space line

The joints share the same $s(t)$ and finish together.

$$\dot{s}_{\max} = \min_i \frac{\dot{q}_{i,\max}}{|\Delta q_i|} = \frac{2}{3}\ \mathrm{s}^{-1}$$

$$\ddot{s}_{\max} = \min_i \frac{\ddot{q}_{i,\max}}{|\Delta q_i|} = \frac{2}{3}\ \mathrm{s}^{-2}$$

Joint 1 determines both limits. Joint 2 need not reach its maximum speed.

Independent fastest profiles would finish at different times.

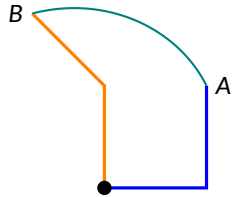
RR solution: a 2.5-second transfer

$$t_r = 1\,\mathrm{s}, \quad t_c = 0.5\,\mathrm{s}, \quad T = 2.5\,\mathrm{s}$$

$$s(t) = \begin{cases} t^2/3, & 0 \le t \le 1, \\ 2t/3 - 1/3, & 1 \le t \le 1.5, \\ 1 - (2.5 - t)^2/3, & 1.5 \le t \le 2.5. \end{cases}$$

t is expressed in seconds.

Peak joint velocities: 60 and $-30°/\mathrm{s}$.
Acceleration magnitudes: 60 and $30°/\mathrm{s}^2$.



The path is straight in joint space. The TCP path is curved: $p(t) = \mathrm{FK}(q(t))$.

From a function to sampled commands

$$t_k = k\Delta t, \qquad q_k = q(t_k)$$

$$\{(t_0, q_0), \ldots, (T, q_f)\}$$

- Include the endpoint exactly at T.
- A smaller step produces more commands.
- Specify what happens between samples.

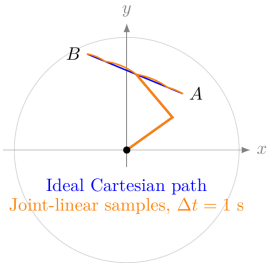
Sampling is part of implementation

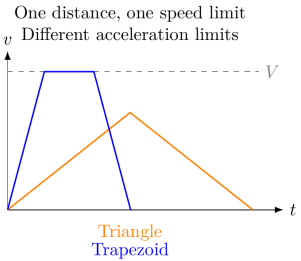
A list of configurations is not yet a complete execution method.

```
times = []
k = 0
while k * dt < T:
    times.append(k * dt)
    k += 1
times.append(T)
```

The last interval may be shorter than Δt.

Sparse samples can change the executed path

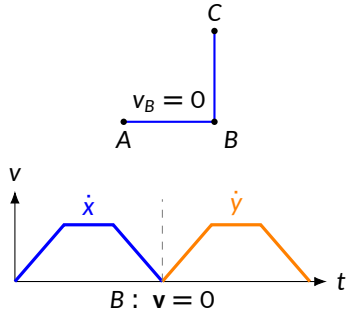




Cartesian mode: compare $\Delta t = 0.05$ and 1 s. Blue: ideal path. Orange: joint-linear interpolation of samples. Checking samples alone can miss a collision between them.

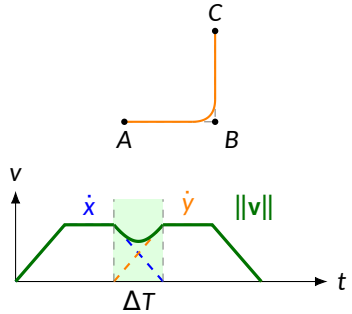
Joining trajectories: stop or blend?

Stop at the waypoint



Two consecutive trapezoids: brake to zero, then accelerate again.

Blend around the waypoint



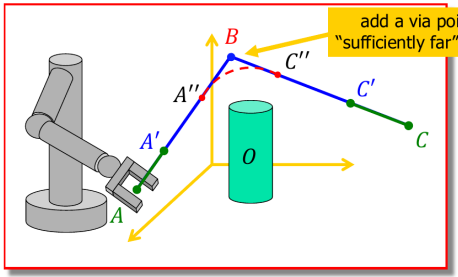
The resultant speed rolls between the two profiles: $\min \|\mathbf{v}\| = V/\sqrt{2} > 0$.

Blue: $\dot{x}$; orange: $\dot{y}$. Green: resultant speed through the blend. Perpendicular segments.

Blending changes the path: check clearance again.

Application example

plan a Cartesian trajectory from A to C (rest-to-rest)
that avoids the obstacle O, with $a \leq a_{\max}$ and $v \leq v_{\max}$



on $\overline{AA'} \rightarrow a_{\max}$; on $\overline{A'B}$ and $\overline{BC'} \rightarrow v_{\max}$; on $\overline{C'C} \rightarrow -a_{\max}$;
+ over-fly between A'' e C'' (e.g., with $a_{\max}$ in norm)

What we will observe on the UR5

1. Joint jogging and TCP movement.
2. Base and tool coordinate frames.
3. Freedrive and stored waypoints.
4. MoveJ versus MoveL.
5. Speed, acceleration and duration.
6. Stopping versus blending.

Same targets → different paths.

Same path → different motion times.

Reference Materials

- A. De Luca — 13 TrajectoryPlanning
- A. De Luca — 06 ProgrammingArchitectures
- O. Bulichev — Tactile sensing experiments with a UR manipulator
- A. Klimchik — L13 Trajectory planning
- A. De Luca — 14 TrajectoryPlanningCartesian
- Universal Robots — Programming and communication interfaces
- Universal Robots — Motion primitives
- Universal Robots — Teaching and manual movement
- Kavraki Lab — Sampling-based motion planning in OMPL
- NVIDIA — cuRobo motion generation

Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521